SCSJ 3483: Web Technology



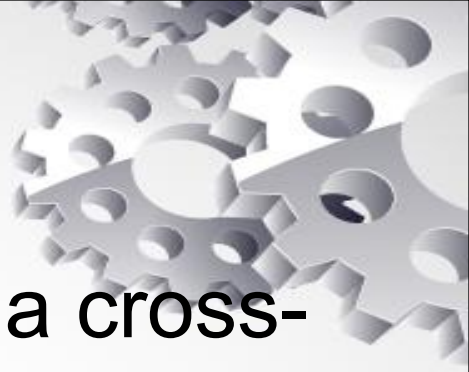
HTML Document Object Model (DOM)

Objectives



- Understand the hierarchical structure of HTML documents represented as a tree of objects
- Understand elements, attributes, text nodes, and their relationships within the document
- Able to manipulate content and structure of web pages dynamically using JavaScript (adding, removing, modifying elements, attributes, and text content)
- Able to handle user events (mouse clicks, keyboard input, mouse movements, form submissions)

What is HTML DOM

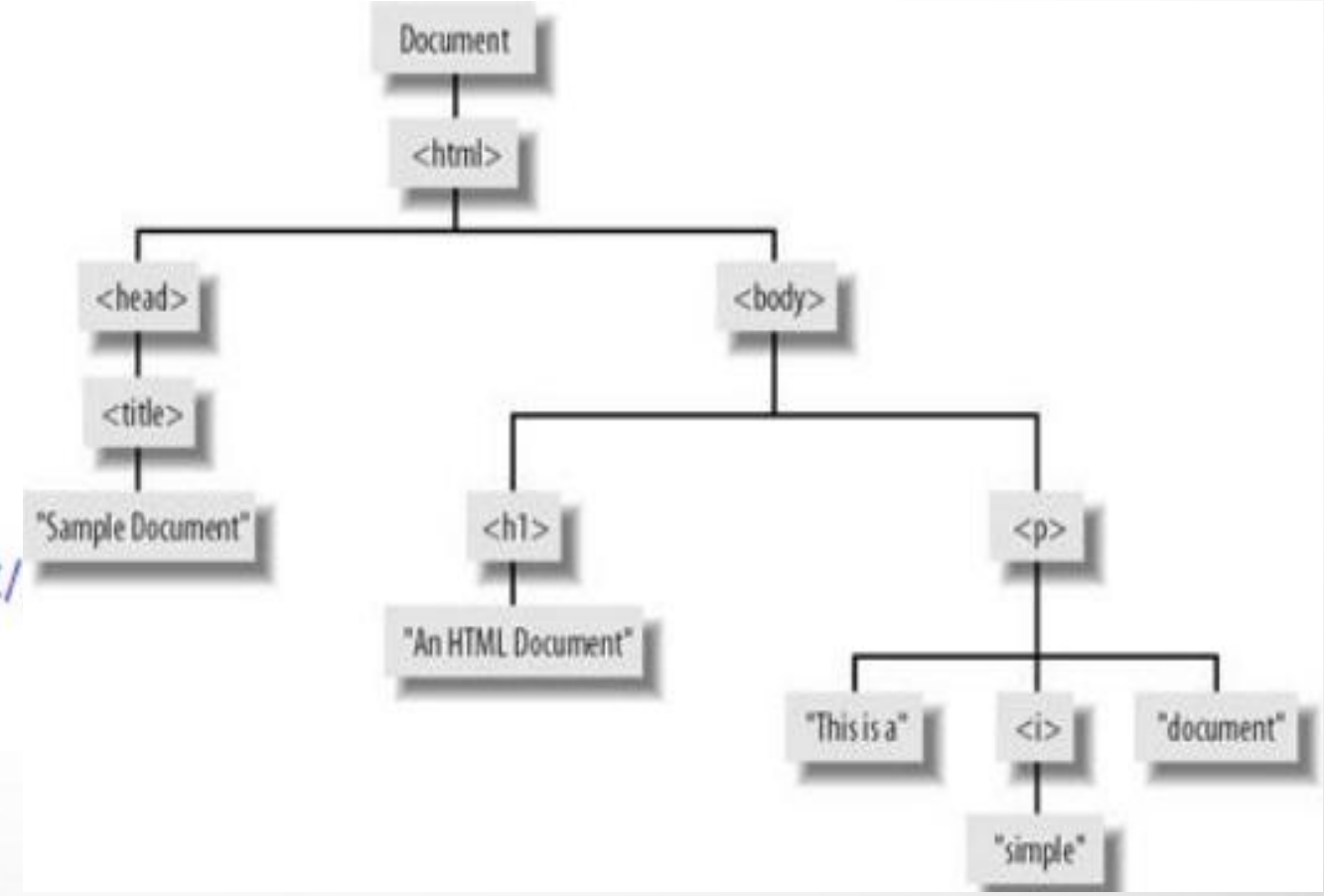


- Definition: The Document Object Model (DOM) is a cross-platform and language-independent interface that treats an XML or HTML document as a tree structure.
- Tree Structure: Each node of the tree represents a part of the document (e.g., elements, attributes, text).
- Relationship: The DOM represents the logical structure of the document and provides a way (ie JavaScript) to interact with and manipulate its content.

Example

Example: A simple HTML document and its DOM Tree representation:

```
<html>
    <head>
        <title>Sample Document</title>
    </head>
    <body>
        <h1>An HTML Document</h1>
        <p>This is a <i>simple</i> document</
    </body>
</html>
```



Activity #1:

Draw DOM Tree representation for the following HTML

DOM Tree:

HTML Code

```
<html>
<head>
    <title>Activity 1</title>
</head>
<body>
    <h1>Hello, World!</h1>
    <p>This is a simple HTML document.</p>
    <li>Item 1.</li>

    <h6>Web Tech 2024</h6>
</body>
</html>
```

Activity #2

HTML Code

```
<html>

<head>
    <title>Activity 1</title>
</head>

<body>
    <h1 style="color: blue;">Hello, World!</h1>
    <div>
        <p>This is a simple HTML document.</p>
        <li>Item 1.</li>
    </div>
    <h6>Web Tech 2024</h6>
</body>

</html>
```

DOM Tree:

Activity #3

HTML Code

```
<head>
    <title>Activity 3</title>
</head>
<body>
    <div class="container">
        <div class="logo">
            <p>logo</p>
        </div>
        <div class="navigasi">
            <a href="">Home</a>
            <a href="">About</a>
        </div>
        <div class="header">
            <p>header /banner</p>
        </div>
        <div class="container-2">
            <div class="side-bar">
                <p>side bar</p>
            </div>
            <div class="body-area">
                <p>body area</p>
                <div class="btn">
                    <a href="">Click me</a>
                </div>
            </div>
        </div>
    </div>
</body>
</html>
```

DOM Tree:



Topics:

1. DOM Selection
2. DOM Manipulation
3. DOM Traversal
4. Event Handling

Exercise/Assignment

1: DOM Selection



Identifying and retrieving specific elements or nodes from the DOM within an HTML document using JavaScript

The initial step, before ie manipulate, traversal, handle event etc...

Once elements are selected from the DOM, developers can manipulate their properties, attributes, styles, and content, or attach event listeners to handle user interactions.

Can be based on element ID, class name, tag name, or more complex selectors based on CSS selectors.

Selecting Elements in Detail

DOM Selection Method	Return Type	Code Example
getElementById()	Single Element	const element = document.getElementById('elementId');
getElementsByClassNam e()	Collection of Elements	const elements = document.getElementsByClassName('class Name');
.getElementsByTagName ()	Collection of Elements	const elements = document.getElementsByTagName('tagNa me');
querySelector():	Single Element.	const element = document.querySelector('selector');
querySelectorAll():	NodeList of Elements.	const elements = document.querySelectorAll('selector');

Example:

```
<head>
  <title>DOM Selection Example</title>
</head>
<body>
<div id="container">
  <h1>Welcome to DOM Selection Example</h1>
</div>
<script>
  // Selecting an element by ID
  var container = document.getElementById('container');
  // Selecting an h1 element
  var heading = container.querySelector('h1');

  // Log the selected elements
  console.log('Selected Container:', container);
  console.log('Selected Heading:', heading);
</script>
</body>
</html>
```



exercise/activity:



- Activity Week 02 : Dom + Js (google shared doc link here)

2: DOM Manipulation



Process of programmatically modifying structure, content, or style of elements within the DOM.

Can be categorized into:

1. **Element manipulation :** deals directly with modifying the properties, attributes, styles, and content of existing HTML elements
2. **Node manipulation :** deals with the structural aspects of the DOM tree, including creating, removing, and replacing nodes, which can be elements, text nodes, or other types of nodes

2.1: Element Manipulation



Involves directly **modifying existing** HTML elements in the DOM.

Example: Changing the text content of a element,

Adding or removing classes from an <input> element.

Purpose: Modify the properties, attributes, and content of existing HTML elements

Methods: Typically involves methods and properties directly related to individual elements.

- setAttribute(): Sets the value of an attribute for an element.
- removeAttribute(): Removes an attribute from an element.
- classList: Provides methods for adding, removing, toggling, and checking classes.
- style: Allows setting inline CSS styles for an element.
- innerHTML: Sets or retrieves the HTML content of an element.

Example:

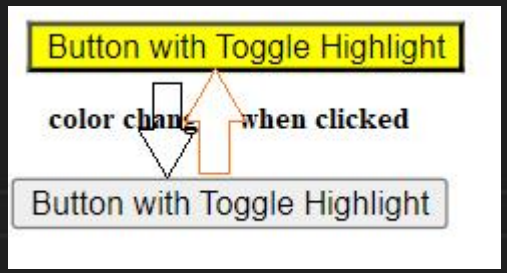
```
<html lang="en">

<head>
    <meta charset="UTF-8">
    <title>Element Manipulation Example</title>
    <style>
        .highlight {
            background-color: yellow;
        }
    </style>
</head>

<body>
    <button id="myDiv" class="highlight" onclick="toggleHighlight()">Button with Toggle Highlight</button>

    <script>
        function toggleHighlight() {
            var divElement = document.getElementById('myDiv');
            divElement.classList.toggle('highlight');
        }
    </script>
</body>

</html>
```



Example

```
7   <body>
8     <input type="text" id="textInput" placeholder="Enter text">
9     <button onclick="addContent()">Overwrite Content</button>
10    <h3 id="contentArea">Content</h3>
11
12    <script>
13      function addContent() {
14        var inputElement = document.getElementById('textInput');
15        var text = inputElement.value;
16        document.getElementById('contentArea').innerText = text;
17        inputElement.value = '';
18      }
19    </script>
20  </body>
```



2.2: Node Manipulation



- Involves working with the nodes of the DOM tree, which includes elements, text nodes, comments, etc.

  Example:

  – Create/appending new nodes, moving nodes around in the tree, and delete/removing existing nodes.

  Purpose: Manipulate the structure of the DOM tree by adding, removing, or replacing nodes.

  Methods: Typically involves methods for working with the DOM tree structure.

  – createElement(): Creates a new element node.
  – createTextNode(): Creates a new text node.
  – appendChild(): Appends a child node to the end of a parent node's list of children.
  – insertBefore(): Inserts a node before a specified child node in the parent's child node list.
  – removeChild(): Removes a child node from its parent node.
  – replaceChild(): Replaces a child node with a new node.

Node Manipulation (insert new node)



4 Steps:

1. Use document.createElement() to create a new element node

```
const newNode = document.createElement('p');
```

2. Use document.createTextNode() to create a new text node if necessary

```
newNode.textContent = "This is the content";
```

3. Select the parent node where you want to insert the new node

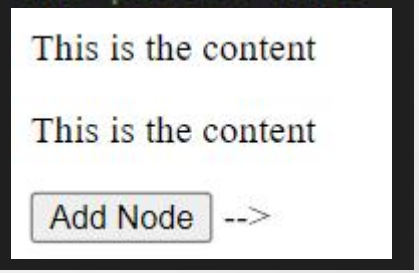
```
const parent = document.getElementById('parent');
```

4. Use appendChild() or insertBefore() to insert the new node into the DOM

```
parent.appendChild(newNode);
//or use insertBefore
//parent.insertBefore(newNode, parent.firstChild);
```

Example

```
5     <title>Add Node to DOM Example</title>
6   </head>
7   <body>
8     <div id="parent"></div>
9     <button onclick="addNode()">Add Node</button>
10
11    <script>
12      function addNode() {
13        // Step 1: Create the node
14        const newNode = document.createElement('p');
15        //set the value/content of the new node (statically for simplicity)
16        newNode.textContent = "This is the content";
17        // Step 3: Select the parent node where the new node will be inserted
18        const parent = document.getElementById('parent');
19        // Step 4: Insert the new node into the DOM by appending it to the parent node
20        parent.appendChild(newNode);
21
22      }
23    </script>
```



Example:

```
7   <body>
8     <ul id="myList">
9       <li>Item 1</li>
10      <li>Item 2</li>
11    </ul>
12    <button onclick="addItem()">Click to Add Item</button>
13
14    <script>
15      function addItem() {
16        var newListElement = document.createElement('li');
17        newListElement.textContent = 'New Item';
18        var list = document.getElementById('myList');
19        list.appendChild(newListElement);
20      }
21    </script>
22  </body>
```



3: DOM Traversal



- 3.1. Traversing Elements
  - Parent Node: const parent = element.parentNode;
  - Child Nodes: const children = parent.childNodes;
  - Next Sibling: const nextSibling = element.nextSibling;
  - Previous Sibling: const prevSibling = element.previousSibling;

Traversing DOM :



DOM traversal : navigating through the hierarchical structure of web documents

- to access, manipulate, and interact with DOM elements.

Understanding DOM traversal techniques is crucial for effectively targeting and manipulating elements within the DOM tree.

Various relationship, methods and properties are available to facilitate DOM traversal, allowing developers to move between different elements within the DOM tree

Allow developers to navigate through the DOM tree, accessing and manipulating elements as needed.

By mastering DOM traversal techniques, developers can efficiently interact with the structure of web documents, enabling the creation of dynamic and interactive web applications.

List of common relationships and methods used in DOM traversal:



Parent-Child Relationship:

- **parentNode:** Returns the parent node of a specified element.
- **childNodes:** Returns a NodeList of child nodes for a specified element.
- **firstChild:** Returns the first child node of a specified element.
- **lastChild:** Returns the last child node of a specified element.

Sibling Relationship:

- **nextSibling:** Returns the next sibling node of a specified element.
- **previousSibling:** Returns the previous sibling node of a specified element.

cont...



Descendant Relationship:

- **querySelector:** Returns the first element within the document that matches the specified CSS selector.
- **querySelectorAll:** Returns a static NodeList representing a list of elements within the document that match the specified group of selectors.

Ancestor Relationship:

- **closest:** Returns the closest ancestor of the current element (or the current element itself) that matches the specified selector.

cont...



Traversal Methods:

- **Traversal with for loops:** Iterating over child nodes using childNodes and traversing through them with loops.
- **Recursion:** Using recursive functions to traverse deeply nested DOM structures efficiently.
- **Traversal with parent, child, and sibling properties:** Directly accessing parent, child, and sibling nodes using properties like parentNode, childNodes, firstChild, lastChild, nextSibling, and previousSibling.

Example:



```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Manual DOM Traversal Example</title>
</head>
<body>

<div class="navbar">
    <a href="#">Home</a>
    <a href="#">About</a>
    <a href="#">Contact</a>
</div>

<div id="main-content" class="content">
    <p>This is paragraph 1.</p>
    <p>This is paragraph 2.</p>
    <p>This is paragraph 3.</p>
    <ul>
        <li>List item 1</li>
        <li>List item 2</li>
        <li>List item 3</li>
    </ul>
```

```
    <div id="target">Target Element</div>
    <input id="textInput1" type="text" placeholder="Enter your text">
    <button id="button1">Button 1</button>
    <button id="button2">Button 2</button>
</div>

<div class="footer">
    &copy; SECJ 3483: Web Technology. All rights reserved.
</div>

<script></script>

</body>
</html>
```

```
<script>
//Manual traversal to the grandparent element
  // Step 1: Select the target element
  var targetElement = document.getElementById('target');

  // Step 2: Navigate to the parent element
  var parentElement = targetElement.parentNode;

  // Step 3: Navigate to the grandparent element
  var grandparentElement = parentElement.parentNode;

  // Step 4: Log the grandparent element
  console.log(grandparentElement);      //the result?
</script>
```

4: DOM Event Handling



Allows us to respond to user interactions and actions on a web page

Event sources: mouse (clicks, double-click, hover), keyboard input, form submissions, etc...

Event Types: click, mouseover, keydown, submit, change, etc...

Event Listeners: functions that wait for a specific event to occur on an element and then execute in response to that event.

Event Handling: involves attaching event listeners to HTML elements, to listen for specific events

Event Propagation and Bubbling: Events in the DOM can propagate or bubble up through the DOM hierarchy from the target element to its ancestors. Can be controlled using methods ie *stopPropagation()*

Set Up Event Handling



The 3 common main ways to set up event handling for an HTML element:

1. Using HTML Attributes
2. Using DOM Event Properties
3. Using addEventListener()

1. Using HTML Attributes:

Directly assign event handlers to HTML elements using event handler attributes like onclick, onmouseover, onkeydown, etc.

This approach directly embeds JavaScript code into the HTML markup

Not recomended.

```
30    <button onclick="handleClick()">Click Me</button>
31    <!-- The onclick attribute is used to assign the handleClick() function
32          as the click event handler for the button element. -->
```

2. Using DOM Event Properties:

- Assign event handlers directly to DOM event properties of an element using JavaScript.
- These properties are prefixed with "on", followed by the name of the event (e.g., onclick, onmouseover, onkeydown).

```
<!-- html -->
<button id="myButton">Click Me</button>

<script>
  document.getElementById('myButton').onclick = function() {
    alert('Button clicked!');
    // the onclick property of the button element is directly
    //assigned a function to handle the click event<script>  };
</script>
```

3. Using addEventListener() :

- Preferred and most flexible method for setting up event handlers.
- Allows to attach event listeners to an element, specifying the event type and a callback function to be executed when the event occurs.
- **Advantages :** better separation of JavaScript code from HTML markup, able to add multiple event listeners to the same element, and more flexibility in managing event handling

```
<!-- html -->
<button id="myButton">Click Me</button>

<script>
  document.getElementById('myButton').addEventListener('click', function() {
    alert('Button clicked!');
  });
</script>
```

Example:

```
<!-- html -->
<input id="textInput1" type="text" placeholder="Enter your text">
<button id="myButton">Button 1</button>

<script>
    // Add event listener to myButton
    document.getElementById("myButton").addEventListener("click", function() {
    //read the value in txtInput1
    const inputValue = document.getElementById("textInput1").value;
    //create new element, li, and append the li
    const listItem = document.createElement("li");
    listItem.textContent = inputValue;
    document.querySelector("ul").appendChild(listItem);
});
```

